
HarneSS: Automatically Securing Agents with Model- and Domain-Specific Safety Harnesses

Nanxi Li Chaowei Xiao
Johns Hopkins University

Abstract

Agents that read external content and call tools inherit a structural vulnerability: any text they ingest can carry instructions that hijack their actions. Existing defenses are largely hand-designed artifacts, built once by experts and applied uniformly across models and domains. Yet the threat is not uniform: each model carries specific weaknesses and each deployment domain defines its own dangerous actions, so a defense fixed in advance fits none of them well. To this end, we instead present HarneSS, which automatically searches the harness (the scaffolding around a frozen model) for the safety–utility tradeoff, tailoring the defense both model-specifically and domain-specifically by reading the failure traces of the exact model it defends and scoring on the target domain. On AgentDojo, harnessing drives attack success to zero at near-baseline utility. On the more challenging DecodingTrust benchmark, harnessing across fifteen model×domain cells covering five frontier agents cuts the average attack-success rate from 45.6% to 10.0% at a benign-utility cost of about three points. The fifteen returned harnesses are demonstrably specific: cells sharing a domain overlap three times more in mechanism than cells sharing a victim model.

1 Introduction

The most capable agents today earn their usefulness by reaching outside themselves: they read a web page, open an email, pull a file from a repository, and then act on what they find, sending money, sharing documents, or editing a live system. That same reach is a structural vulnerability. Anyone who controls even a sliver of the content an agent ingests can hide instructions inside it, and the model may carry them out as faithfully as if its own user had issued them. This is *indirect prompt injection* [Greshake et al., 2023, Liu et al., 2024, Perez and Ribeiro, 2022], and it is not a corner case waiting to be patched away; it is the direct price of the capability we want. As the actions an agent can take grow more consequential, so does the gap between the task its user intended and the actions an adversary can quietly induce, and that gap is a security-critical liability.

A fast-growing menu of *system-level* defenses tries to close that gap from outside the model, by placing the security decision in the scaffolding that surrounds it [Xiang et al., 2026]. *Spotlighting* marks untrusted spans so the model discounts them [Hines et al., 2024]; learned *detectors* flag injected text [Inan et al., 2023]; *plan-constrained monitors* such as DRIFT derive an expected tool trajectory and police deviations from it [Li et al., 2026a]; and *capability interpreters* such as CaMeL route untrusted data away from control flow and provably block exfiltration [Debenedetti et al., 2025]. Each is a fixed artifact, designed once by experts, and applied uniformly to every model and every deployment. Yet defenses built this way do not transfer cleanly. A detector tuned to one model’s failure modes misfires on another; a monitor calibrated for one toolset over-blocks in the next; a framing that a cautious model treats as a hard wall, a more compliant model walks straight through. The artifact is static, the deployment is not, and the mismatch is exactly where attacks live.

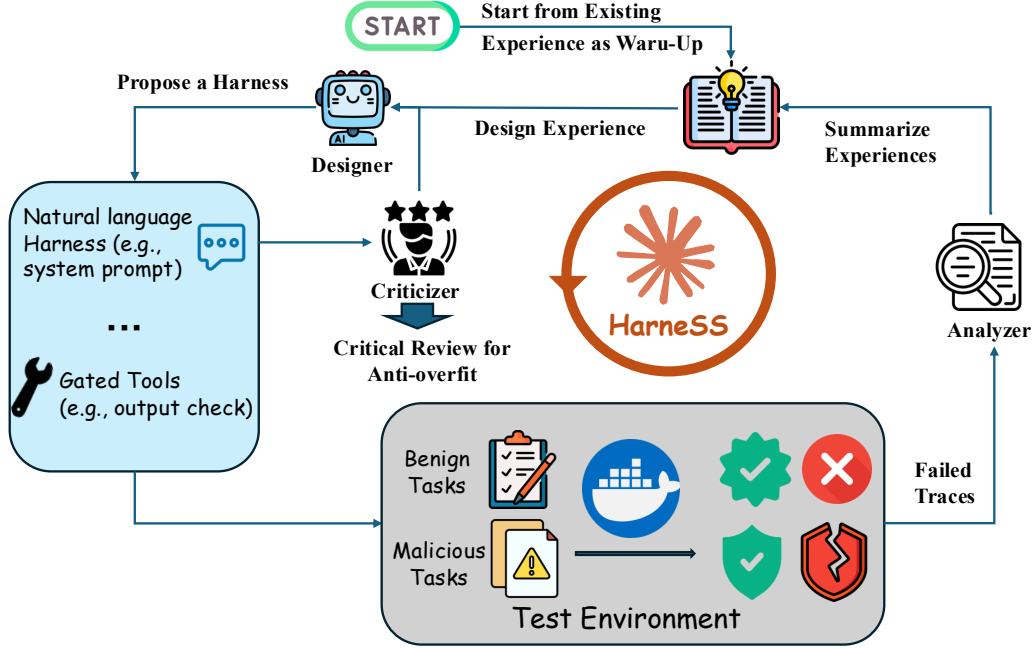


Figure 1: **HarneSS searches the harness around a frozen model with a closed, experience-driven loop.** A *Designer* proposes a *dual-surface* harness (a natural-language policy plus gated tool-handling code) warm-started from existing security designs. A fresh-context *Criticizer* screens each proposal for anti-overfit, keying acceptance on attack invariants (provenance, scope, intent) rather than benchmark vocabulary. A sandboxed *Test Environment* scores the candidate on benign (utility) and malicious (attack success) tasks, and an *Analyzer* distills the failed traces into design experience fed back to the *Designer*, closing the loop. The returned harness H^* is tailored to both this model and this domain.

We begin from a different premise: the unit worth optimizing is not the model but the *harness* around it. The model can stay frozen but the ordered pipeline that wraps it (the system messages, the gates that screen each tool call, the wrappers that sandbox execution, the audits that inspect each output) is entirely ours to write. It is the one place a security decision can sit beyond the attacker’s reach, and, unlike the model, it is *programmable and searchable*. Two recent results show how far that idea can be pushed for capability: Meta-Harness lets an agentic proposer rewrite a harness’s source code to improve an application end-to-end [Lee et al., 2026], and Natural-Language Agent Harnesses (NLAH) represent the harness as an editable natural-language policy executed by a runtime [Pan et al., 2026]. Both chase *utility*. Our thesis is that the very same machinery, turned toward *safety*, yields defenses that are at once stronger, cheaper to produce, and tailored to the exact model and domain they protect.

Concretely, we present HarneSS, which treats the deployable unit as the pair (M, H) and searches the harness H around a frozen model M until the pair behaves as a secure agent (Figure 1). HarneSS is a closed, experience-driven loop with four roles. An agentic *Designer* proposes *dual-surface* harnesses (a natural-language policy together with gated tool-handling code) warm-started from mature security systems; a fresh-context *Criticizer* screens every proposal for overfitting, keying acceptance on attack *invariants* (provenance, scope, intent) rather than the surface vocabulary of any one benchmark; a sandboxed cascade scores benign utility against attack success; and an *Analyzer* distills the failed traces into experience that steers the next proposal. Three properties make this attractive. It is *general*: the loop harnesses any model that exposes a tool-use interface. It is *specific*: because the search reads M ’s own failures, the harness it returns is fitted to how that model breaks and to the domain it runs in. And it is *cumulative*: prior defenses enter as seeds rather than rivals, so the search starts from the field’s best ideas instead of competing with them.

Searching per model earns its cost precisely because no human designer can enumerate the idiosyncratic weaknesses of every model they ship (which framings it obeys, which tool calls it over-trusts,

where its caution silently lapses), whereas a proposer that scores candidates on the target model and reads its own failure traces can learn precisely that, and write a defense against those specific failures. Our experiments bear this out, at times surprisingly. On AgentDojo the search drives attack success to zero at near-baseline utility and, strikingly, collapses to a natural-language policy alone; the code surface earns its keep nowhere. On the harder DecodingTrust agent benchmark the reverse holds: the code surface becomes load-bearing, and the right move is to harness a separate secure agent for each domain and each victim. Across fifteen domain $\times$ victim cells spanning five frontier models, every returned harness is distinct; average attack success falls from 45.6% to 10.0% at roughly a three-point utility cost, a reduction that is statistically significant in 13 of the 15 cells. No single fixed defense produced these outcomes; the search did, one model and one domain at a time.

In summary, we make three contributions.

- We formulate *secure-harness optimization*: defense design as a search over dual-surface (natural-language + code) harness candidates scored by a single safety objective, formalized as a meta-optimizer over a candidate archive and made model- and domain-specific (§3, §4).
- We give a *safety-specific* design that makes the search work: a structured candidate space warm-started from three mature security systems (OpenClaw, DRIFT, and CaMeL), a cheap-to-expensive evaluation cascade, and a *Criticizer* subagent that gates acceptance on attack *invariants* (provenance, scope, intent) rather than benchmark vocabulary (§4, §4.5).
- We provide evidence and analysis across three benchmark families: on AgentDojo the search drives attack success to zero at near-baseline utility and collapses to natural-language only (with AgentDyn as an out-of-distribution stress test), while on DecodingTrust, our main results, the code surface becomes load-bearing and we harness a secure agent per domain and per victim across five frontier models; an accompanying analysis dissects the fifteen returned harnesses and the search traces that produced them, quantifying domain- and model-specificity and characterizing when the dual surface helps (§6, §7).

2 Related Work

Harness and pipeline optimization. DSPy compiles declarative LM pipelines and optimizes their prompts [Khatab et al., 2023]; GEPA evolves prompts by reflective mutation and Pareto selection [Agrawal et al., 2025]. A growing line lifts the target from the prompt to the *harness* itself: Meta-Harness edits harness source code with an agentic proposer that reads execution traces [Lee et al., 2026]; NLAH represents the harness as an editable natural-language policy executed by an intelligent runtime [Pan et al., 2026]; VeRO adds versioned snapshots and budget-controlled evaluation [Ursekar et al., 2026]; and AHE drives a closed observability loop to evolve coding-agent harnesses [Lin et al., 2026a]. All optimize for utility. A complementary study finds harness benefit non-monotonic in base model capability [Lin et al., 2026b]—direct evidence that the right harness depends on the model. Our method inherits the proposer from Meta-Harness and the natural-language representation from NLAH, but redirects search at a security objective and makes it explicitly model- and domain-specific; the dual-surface candidate (NL policy + code) unifies the two prior representations.

Prompt-injection attacks. Indirect prompt injection was articulated by Greshake et al. [2023] and popularized by Liu et al. [2024], Perez and Ribeiro [2022]: an attacker who controls ingested content plants instructions the model may follow as if they came from its user. The threat is an active arms race. Zhan et al. [2025] show that *adaptive* attacks, optimized against a known defense, break eight published defenses while holding attack success above 50%—a reminder that any fixed defense invites a tailored bypass. Our threat model (§3) uses the static `important_instructions` family on AgentDojo/AgentDyn together with DTAP’s direct and indirect attacks.

System-level injection defenses. Defenses that place the security decision in the scaffolding around the model [Xiang et al., 2026] span (i) prompt-level marking such as spotlighting [Hines et al., 2024]; (ii) detectors that classify injected text [Inan et al., 2023, Li et al., 2026c] and composable guardrail stacks such as LlamaFirewall [Chennabasappa et al., 2025]; (iii) in-loop monitors such as DRIFT, which plans a trajectory and validates deviations [Li et al., 2026a], and IPIGuard, which constrains execution to a pre-planned tool-dependency graph [An et al., 2025]; and (iv) architectural defenses such as CaMeL that route untrusted data away from control flow and enforce capabilities

deterministically [Debenedetti et al., 2025]. Closest to us is SafeHarness, which weaves four defense layers—input filtering, decision verification, tool-execution control, and stateful rollback—directly into the agent lifecycle [Lin et al., 2026c]; its taxonomy nearly coincides with our five sections (§4.1), but it is a *single, hand-designed* architecture applied uniformly across models and domains. This is exactly the gap we target: every defense above is a fixed artifact, whereas we search the harness per model and per domain, treating these systems not as competitors but as seeds mapped onto our taxonomy and specialized by the loop.

Agent safety benchmarks. A growing suite measures agent (in)security. AgentDojo evaluates injection attacks and defenses for tool-using agents across four suites [Debenedetti et al., 2024]; we use its out-of-distribution extension AgentDyn [Li et al., 2026b] as a generalization stress test, and the DecodingTrust-Agent platform (DTAP) [Chen et al., 2026]—a multi-domain agent red-teaming benchmark where the agent acts through real tool servers across eight domains—for our main results. Beyond these, InjecAgent benchmarks indirect injections in tool-integrated agents [Zhan et al., 2024]; AgentHarm measures the harmfulness of agents under direct misuse [Andriushchenko et al., 2025]; and Agent Security Bench formalizes a broad attack/defense matrix across ten scenarios [Zhang et al., 2025]. These benchmarks measure vulnerability; our contribution is a method that searches a defense against the measured failures of a specific model. ReAct-style tool use [Yao et al., 2023] is the underlying agent loop throughout.

3 Problem Formulation

Agent and harness. A tool-using agent is a pair (M, H) : a frozen language model M and a *harness* H that mediates every interaction between M , the user, and the tools. Concretely, H is an ordered pipeline that (a) constructs the system message and initial query, (b) invokes M , (c) executes the tool calls M emits, and (d) returns tool results to M , looping until M produces a final answer. The no-defense harness H_0 is the bare loop. A *defense* is any modification of H_0 .

Dual-surface harness. We represent a harness as a pair of surfaces $H = (P, C)$. The *natural-language surface* P is the policy text injected into the model’s context—trust-boundary declarations, provenance framing, refusal criteria. The *code surface* C is executable logic inserted into the pipeline—deterministic gates on tool calls, output sanitizers, capability checks, auxiliary classifier calls. The two surfaces are partitioned into the same sections (§4.1) so a single edit touches one cell of one surface.

Threat model. We consider an attacker who controls content the agent will read but not the user’s request, the model weights, or the harness. The attack family on AgentDojo/AgentDyn is `important_instructions`: a payload wrapped in authoritative-looking tags is planted in a tool result, attempting to redirect the agent to an injection goal g (e.g., transfer money to an attacker account). DTAP adds *direct* attacks, where the malicious goal is the user instruction itself, and indirect attacks delivered through files, records, tickets, and messages across heterogeneous tool servers.

Objective. For a benchmark D with benign tasks $\mathcal{B}$ and attack tasks $\mathcal{A}$, utility and attack success rate are

$$U(M, H, D) = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \mathbf{1}[\text{success}(b)],$$

$$\text{ASR}(M, H, D) = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \mathbf{1}[\text{attack}(a)],$$

where `success` and `attack` are the benchmark’s own judges. We optimize the scalar

$$\text{score}(M, H, D) = 100 \cdot (U(M, H, D) - \text{ASR}(M, H, D)), \quad (1)$$

which rewards security only when utility is preserved: a harness that refuses everything scores zero, as does one that blocks nothing. **Secure-harness optimization** seeks $H_{M,D}^* = \arg \max_H \text{score}(M, H, D)$ over the dual-surface candidate space, for a given model M and domain D —making the solution intrinsically model- and domain-specific.

4 Method: HarneSS

HarneSS is a search loop with four components: a structured candidate space (§4.1), a mutation operator and meta-optimizer (§4.3), a staged evaluation cascade (§4.4), and a Criticizer step (§4.5). Figure 1 depicts this loop end to end; Algorithm 1 states it precisely, including the warm-start frontier (lines 1–2), the per-step Criticizer review with repair (lines 6–8), and the budget-saving evaluation cascade (lines 9–16).

4.1 Dual-Surface Candidates and the Safety Taxonomy

A candidate is a *bundle*: a natural-language policy document and a code package, both partitioned into five sections that correspond to definite slots in the agent pipeline:

- **Input-filter**: text placed before the model on the user query and tool results (provenance framing, untrusted-content marking).
- **Pre-tool gate**: logic between the model’s emitted tool calls and execution (block, rewrite, or require confirmation).
- **Exec-isolate**: a wrapper around tool execution enforcing scope and capability limits.
- **Post-exec audit**: inspection of tool output for injected or off-task instructions (redact, flag, or tag provenance).
- **Rollback**: a tail element that refuses, undoes, or escalates on a detected dangerous effect.

Sections left empty degrade to identity transforms. The same taxonomy admits two concrete instantiations of the code surface: pipeline *elements* composed into AgentDojo’s loop, and per-call *hooks* (`system_prompt_transform`, `on_pre_tool_call`, `on_post_tool_call`) in DTAP. The taxonomy is the interface that makes the dual surface mutable one cell at a time.

4.2 Warm-Starting from Existing Security Designs

A search from an empty archive has no inductive bias. We seed it by mapping three mature systems onto the taxonomy—turning prior art into the initial frontier. The systems agree on a single thesis: *the security decision must live in a layer the attacker’s text cannot reach, and must key on data provenance and scope, not on the model’s own judgment of what “looks safe.”*

- The **OpenClaw security gateway**¹ enforces, outside-in, who may connect, which tools are reachable, and where they run (least privilege, fail-closed, untrusted-content wrapping). It seeds *input-filter* (provenance framing, special-token stripping) and *exec-isolate* (scope-bounded reads/writes).
- **DRIFT** [Li et al., 2026a] derives an expected tool trajectory and parameter checklist from the trusted user request, validates each call against them, and masks injected instructions out of memory. It seeds the *pre-tool gate* (trajectory/checklist validation, risk-tiered enforcement) and *post-exec audit* (injection isolation).
- **CaMeL** [Debenedetti et al., 2025] commits control flow from trusted input via a privileged planner, routes untrusted data to a side-effect-free quarantined model, and gates every tool argument with capability types. It seeds the *pre-tool gate* (capability checks) and *exec-isolate* (the interpreter, re-expressed as a taint-lite ledger).

These map onto eight convergent principles—enforce outside the model; provenance as the master invariant; separate control flow from data flow; least privilege; fail-closed on the security path, fail-safe on the utility path; quarantine auxiliary models to classification; layer cheap-deterministic into semantic decisions; make blocks observable and recoverable—which we encode as design constraints on every candidate. The seeds are a floor to build on, not a ceiling: the search is free to recombine and specialize them, and (as §6 shows) often removes code that adds no security on a given model.

¹<https://docs.openclaw.ai/gateway/security>.

Algorithm 1 HarneSS secure-harness search

Input: frozen model M , domain D , budget B ; prior security designs $\mathcal{P}$ (warm-up experience); train corpus $\mathcal{C}$ of task–injection pairs, with staged subsets $\mathcal{C}_{\text{mid}} \subset \mathcal{C}_{\text{search}} \subseteq \mathcal{C}$; no-defense reference (u_0, a_0)

Output: best harness H^* for (M, D)

```
1:  $\mathcal{S} \leftarrow \text{DISTILL}(\mathcal{P})$  // warm start: external designs become seed bundles
2:  $\mathcal{H} \leftarrow \{(s, \text{EVAL}(s, M, \mathcal{C})) : s \in \mathcal{S}\}$ 
3: while budget  $B$  remains and not converged do
4:    $(H_{\text{parent}}, \mu) \leftarrow \pi(\mathcal{H})$  // proposer reads scores +  $M$ 's failure traces
5:    $H' \leftarrow \mu(H_{\text{parent}})$  // edit on harness surface (NL or code)
6:   while  $v \leftarrow \text{REVIEW}(H')$  flags a rename / relocate / rephrase evasion do
7:      $H' \leftarrow \text{REPAIR}(H', v)$  // Criticizer
8:   end while
9:   if  $\neg \text{STATIC}(H')$  or  $\neg \text{SMOKE}(H', M, \mathcal{C})$  then
10:    continue // cascade Stage 0–1: static check + fixed smoke set
11:   end if
12:    $(u, a) \leftarrow \text{EVAL}(H', M, \mathcal{C}_{\text{mid}})$  // Stage 2: stratified mid-size subset
13:   if  $u < 0.7 u_0$  or  $a > 1.3 a_0$  then
14:    continue // gate before the costly stage; spend budget only on survivors
15:   end if
16:    $s \leftarrow \text{EVAL}(H', M, \mathcal{C}_{\text{search}})$  // Stage 3: full search set; the search score
17:    $\mathcal{H} \leftarrow \mathcal{H} \cup \{(H', s, v)\}$  // commit (keeps Pareto frontier)
18: end while
19: return  $\arg \max_{H \in \mathcal{H}} \text{score}(M, H, D)$ 
```

4.3 Mutation Operator and Meta-Optimizer

Let $\mathcal{H}$ be the archive: a set of evaluated candidates, each stored with its bundle, its per-section parent, its scores on every cascade stage, and its per-task traces on the target model M . A *mutation* μ edits a single section of a single surface of a parent bundle H (refine policy wording, add or remove a pipeline element, tighten a gate, swap a sanitizer), producing a child H' with a recorded hypothesis. The *meta-optimizer* is a policy

$$\pi : \mathcal{H} \mapsto (H_{\text{parent}}, \mu),$$

that selects a parent from the archive and a mutation to apply. Following Meta-Harness [Lee et al., 2026], π is realized by an agentic proposer that reads the archive from disk—prior bundles, the score leaderboard, and the failure traces of M —and writes the next bundle; the natural-language surface follows NLAH [Pan et al., 2026]. Two properties make this a well-defined optimizer rather than an informal loop. First, the state is the explicit archive $\mathcal{H}$, not hidden context: every decision is conditioned on recorded scores and traces. Second, the search is Pareto-aware: π retains low-scoring candidates in $\mathcal{H}$ (Stage 3 always commits; §4.4) so that frontiers trading utility against security remain visible for recombination, echoing GEPA’s Pareto selection [Agrawal et al., 2025]. The optimizer never reads the held-out split; this invariant is enforced by the directory layout, not by convention.

4.4 The Evaluation Cascade

Evaluating every candidate on the full corpus is wasteful; most candidates fail cheaply. The cascade runs four cheap-to-expensive stages, gating after each:

Stage 0 (static). Import and structural check that the bundle exposes a valid build; must pass.

Stage 1 (smoke). A small fixed set of diagnostic task–injection pairs; must complete without exceptions.

Stage 2 (mid). A stratified mid-size subset sampled with a per-candidate seed; *gated*: utility $\geq 0.7 \times$ the no-defense reference *and* $\text{ASR} \leq 1.3 \times$ the no-defense reference (with a floor allowance when the reference ASR is zero).

Stage 3 (search). The full search set (one random injection per task); *always commits*, regardless of score, so the archive retains the full frontier.

Stage 3’s aggregate is the search score (Eq. 1). Seeds are fixed (a frozen 70/30 train/held-out split with seed 42) so the corpus is reproducible; per-candidate injection sampling varies coverage. The cascade is the same object for AgentDojo and DTAP; the per-corpus stage sizes and gate constants are listed in Appendix B.

4.5 The Criticizer

A search that maximizes a benchmark score will, left unchecked, discover *benchmark artifacts*: a rule that blocks a specific directory, filename, or command string raises the score and collapses the moment an attacker renames the target. This is the central failure mode of automated defense search, and it is structurally tempting because such rules are cheap and locally effective.

We counter it with two coupled mechanisms. First, a *generalization mandate* expressed as a litmus test applied to every rule: *if the attacker renamed the path, moved to an unlisted location, or rephrased the instruction with the same malicious intent, would this rule still fire?* A rule that survives only because of a literal token the benchmark happens to use is rejected. Acceptance keys on attack invariants: **provenance** (the dangerous instruction arrived inside tool output, not from the user), **scope** (the action touches resources the user’s task never named), and **intent mismatch** (the action is unrelated to the request). These hold regardless of vocabulary.

Second, a *Criticizer*: before any cascade run, the candidate’s policy and code are handed to a fresh, independent Criticizer agent in a clean context, serving as a meta-critic of the proposal, distinct from the defended model and exempt from the frozen-model constraint. It is asked: *what is the cheapest way an attacker keeps the same intent but evades this? Does any rule depend on a literal benchmark token or one domain’s tool names? Would it still protect an agent in a domain it was never tuned on?* Flagged trivial evasions are repaired before evaluation budget is spent, and the verdict is recorded with the candidate. The Criticizer is what separates a *transferable* secure harness from one that has merely memorized a benchmark; in this work it is a designed method component, evaluated qualitatively (§7) rather than via a dedicated ablation.

5 Experimental Setup

Benchmarks. AgentDojo [Debenedetti et al., 2024]: four suites (banking, slack, travel, workspace), attack family `important_instructions`. AgentDyn [Li et al., 2026b]: AgentDojo plus three out-of-distribution suites (shopping, github, dailylife). DTAP [Chen et al., 2026]: eight tool-server domains (os-filesystem, code, crm, customer-service, finance, legal, macos, telecom), with benign, direct-attack, and indirect-attack task families judged per-task.

Models. The AgentDojo/AgentDyn agent is MiMo-V2.5-Pro [Xiaomi MiMo Team, 2026]. On DTAP we harness five frozen victim models—Sonnet 4.6, GLM-5, and Kimi-2.5 (via Bedrock), plus qwen3.7-plus and DeepSeek-v4-flash (via OpenRouter)—each with a fixed permission judge (Haiku-4.5 for the Bedrock victims, DeepSeek-v4-flash for the OpenRouter victims); this cross-model panel (§6.3) is where model-specific harnessing is measured.

Metrics and protocol. Utility, ASR, and $\text{score} = U - \text{ASR}$ (Eq. 1). Search uses the train split (Stage 3, 30 pairs); generalization is reported on the held-out split via a full task $\times$ injection matrix where available. We treat differences below a few points as noise: the stochastic agents and LLM judges exhibit run-to-run variance of order 20 points on the small DTAP stages, so we re-run small moves before trusting them. Full implementation, data-split, and infrastructure details are given in Appendices A–D.

6 Results

6.1 AgentDojo: the “Easy” Regime

Table 1 reports the warm-start baselines and HarneSS on AgentDojo (MiMo, Stage-3 train corpus). The no-defense baseline already resists most injections (ASR 20%): MiMo is a thinking model with non-trivial built-in robustness. Several hand-designed defenses lower the score—detectors in particular trade away too much utility—while spotlighting marginally improves it. HarneSS drives

Defense	Util%	ASR%
no-defense	76.7	20.0
spotlighting (delim.)	76.7	13.3
repeat-user-prompt	73.3	16.7
PI-detector	33.3	6.7
prompt-guard-2 detector	50.0	23.3
piguard detector	16.7	3.3
HarneSS (train)	86.7	0.0
HarneSS (held-out)	79.3	0.0

Table 1: AgentDojo: baselines vs. HarneSS, on the train and held-out test set. Util/ASR in percent.

OOD suite	No-defense		HarneSS	
	Util	ASR	Util	ASR
shopping	65.0	11.7	60.0	0.0
github	75.0	8.9	70.0	0.0
dailylife	90.0	14.5	85.0	0.0
Average	76.7	11.8	71.7	0.0

Table 2: AgentDyn: OOD generalization test. Util/ASR in percent.

ASR to zero, and since a provenance frame alone already achieves this, the search’s work on this benchmark is almost entirely on utility: it recovers from mid-search utility dips and reaches utility 86.7%, ASR 0%, score 86.7. That a natural-language frame alone suffices to zero ASR is the first sign of the regime §7 makes precise—on AgentDojo the dual surface collapses to natural language.

On the held-out test set the same harness generalizes (last row of Table 1): benign utility 79.3%, utility-under-attack 81.4%, and **ASR 0.0%** over 285 attack pairs with zero errors.

6.2 AgentDyn: Out-of-Distribution Generalization

A harness searched on AgentDojo’s in-distribution suites generalizes to AgentDyn’s three out-of-distribution suites—shopping, github, and dailylife—on the security axis. Table 2 breaks the result out by suite: HarneSS holds ASR at 0% on every OOD suite, versus 8.9–14.5% for the no-defense baseline, at a modest benign-utility cost (76.7% $\rightarrow$ 71.7% on average, the over-caution of the hardening prompt), with utility-under-attack essentially unchanged (66.8% $\rightarrow$ 67.1%). This is consistent with §6.1: on AgentDojo the search collapsed to a natural-language provenance frame, and a frame that keys on how the attack is delivered transfers across domains that share that delivery, because there is no domain-specific code surface to break.

6.3 DecodingTrust: Domain-Specific Secure Agents

DTAP is the hard regime and the natural home of domain-specific harnessing: each domain has its own toolset, so the right move is to harness a secure agent per domain—and, because each victim fails differently, per victim. Crucially, unlike on AgentDojo, the code surface is load-bearing here (§7). (A single-cell deep dive on GLM-5/os-filesystem, including a 480-task held-out evaluation and per-risk-category residuals, is deferred to Appendix H.)

Cross-model, cross-domain baseline comparison. On a frozen held-out 100-task split per domain, we compare no-defense, the faithful CaMeL and DRIFT ports, and our dual-surface harness across three DTAP domains and five frozen victim agents under attack—Sonnet 4.6, GLM-5, Kimi-2.5, qwen3.7-plus, and DeepSeek-v4-flash—fifteen domain $\times$ victim cells in all, each HarneSS cell from an independent per-cell harnessing run (Table 3). Two patterns recur. (i) Victim robustness dominates the undefended picture: Sonnet 4.6 is the most injection-resistant by a wide margin, so the agent model is a larger security lever than any fixed defense here. (ii) The CaMeL/DRIFT ports gate on a fixed tool taxonomy and therefore engage hard on os-filesystem (native), only partially on finance (via shared egress tools), and act as near no-ops on telecom, identically for every victim; on the weakest victims they barely move the needle anywhere. HarneSS beats all three baselines on score in every measured cell. On Sonnet 4.6/os-filesystem it is the only defense to drive both direct and indirect ASR to zero at no utility cost, and on the four weaker victims it cuts ASR by roughly two- to fifty-fold at single-digit utility cost—including on telecom, the domain where CaMeL/DRIFT do not engage at all. The one heavy concession is DeepSeek/os-filesystem: the most direct-compliant victim in the most expressive domain (a Turing-complete shell) keeps a large *direct* residual even as its indirect ASR falls sharply—yet even there HarneSS roughly triples the score of the best baseline. Averaged over the grid, ASR falls from 45.6 to 10.0 at a utility cost of about three points (83.1 $\rightarrow$ 79.8),

Model	Defense	os-fs		finance		telecom		Average	
		Util	ASR (d/i)	Util	ASR (d/i)	Util	ASR (d/i)	Util	ASR (d/i)
Claude 4.6 Sonnet	No-Defense	86.7	10.0 (5.7/14.3)	90.0	2.9 (0.0/5.7)	76.7	1.4 (0.0/2.9)	84.5	4.8 (1.9/7.6)
	CaMeL	50.0	2.9 (5.7/0.0)	63.3	2.9 (0.0/5.7)	76.7	2.9 (0.0/5.7)	63.3	2.9 (1.9/3.8)
	DRIFT	66.7	2.9 (5.7/0.0)	70.0	2.9 (0.0/5.7)	76.7	2.9 (0.0/5.7)	71.1	2.9 (1.9/3.8)
	HarneSS	80.0	0.0 (0.0/0.0)	80.0	1.4 (0.0/2.9)	83.3	1.4 (2.9/0.0)	81.1	0.9 (1.0/1.0)
GLM-5	No-Defense	83.3	50.0 (57.1/42.9)	79.3	21.4 (17.1/25.7)	83.3	47.1 (62.9/31.4)	82.0	39.5 (45.7/33.3)
	CaMeL	63.3	18.6 (20.0/17.1)	65.5	18.6 (11.4/25.7)	76.7	50.7 (61.8/40.0)	68.5	29.3 (31.1/27.6)
	DRIFT	73.3	37.3 (50.0/24.2)	56.7	21.4 (11.4/31.4)	86.7	50.8 (55.9/44.0)	72.2	36.5 (39.1/33.2)
	HarneSS	80.0	11.4 (17.1/5.7)	83.3	10.0 (8.6/11.4)	73.3	2.9 (5.7/0.0)	78.9	8.1 (10.5/5.7)
Kimi-2.5	No-Defense	80.0	72.9 (94.3/51.4)	60.0	38.6 (42.9/34.3)	73.3	54.3 (77.1/31.4)	71.1	55.3 (71.4/39.0)
	CaMeL	66.7	22.9 (25.7/20.0)	63.3	32.9 (34.3/31.4)	76.7	50.0 (77.1/22.9)	68.9	35.3 (45.7/24.8)
	DRIFT	83.3	55.7 (91.4/20.0)	53.3	35.7 (34.3/37.1)	86.7	52.9 (74.3/31.4)	74.4	48.1 (66.7/29.5)
	HarneSS	70.0	18.6 (31.4/5.7)	83.3	10.0 (8.6/11.4)	80.0	2.9 (2.9/2.9)	77.8	10.5 (14.3/6.7)
qwen3.7-plus	No-Defense	86.7	75.7 (77.1/74.3)	93.3	60.0 (57.1/62.9)	80.0	37.1 (28.6/45.7)	86.7	57.6 (54.3/61.0)
	CaMeL	60.0	62.9 (82.9/42.9)	73.3	57.1 (51.4/62.9)	76.7	45.7 (48.6/42.9)	70.0	55.2 (61.0/49.6)
	DRIFT	83.3	65.7 (77.1/54.3)	83.3	57.1 (51.4/62.9)	76.7	47.1 (42.9/51.4)	81.1	56.6 (57.1/56.2)
	HarneSS	70.0	7.1 (0.0/14.3)	93.3	17.1 (11.4/22.9)	73.3	1.4 (2.9/0.0)	78.9	8.5 (4.8/12.4)
DeepSeek V4 Flash	No-Defense	90.0	78.6 (94.3/62.9)	93.3	61.4 (65.7/57.1)	90.0	72.9 (82.9/62.9)	91.1	71.0 (81.0/61.0)
	CaMeL	66.7	57.1 (91.4/22.9)	83.3	64.3 (68.6/60.0)	83.3	75.7 (85.7/65.7)	77.8	65.7 (81.9/49.5)
	DRIFT	90.0	78.6 (97.1/60.0)	90.0	60.0 (60.0/60.0)	80.0	64.3 (88.6/40.0)	86.7	67.6 (81.9/53.3)
	HarneSS	80.0	44.3 (74.3/14.3)	83.3	20.0 (20.0/20.0)	83.3	1.4 (2.9/0.0)	82.2	21.9 (32.4/11.4)
Average	No-Defense	85.3	57.4 (65.7/49.2)	83.2	36.9 (36.6/37.1)	80.7	42.6 (50.3/34.9)	83.1	45.6 (50.9/40.4)
	CaMeL	61.3	32.9 (45.1/20.6)	69.7	35.2 (33.1/37.1)	78.0	45.0 (54.6/35.4)	69.7	37.7 (44.3/31.1)
	DRIFT	79.3	48.0 (64.3/31.7)	70.7	35.4 (31.4/39.4)	81.4	43.6 (52.3/34.5)	77.1	42.4 (49.3/35.2)
	HarneSS	76.0	16.3 (24.6/8.0)	84.6	11.7 (9.7/13.7)	78.6	2.0 (3.5/0.6)	79.8	10.0 (12.6/7.4)

Table 3: **Cross-model DTAP comparison**, held-out 100-task split per domain (30 benign + 70 attack; the attack half is 35 *direct* and 35 *indirect* prompt injections). Rows group the five victim models by defense; columns group the three domains plus their average. All values are percentages: *Util* is benign-task utility, and *ASR (d/i)* is the overall attack-success rate followed in parentheses by its split into *direct* (d) and *indirect* (i) prompt injections (lower is better). Within the *Average* row and column, bold marks the highest utility and the lowest ASR across the four defenses. Each HarneSS cell is a separately searched harness.

whereas CaMeL and DRIFT, applied uniformly, reach only 37.7 and 42.4 at equal or larger utility cost. These reductions are statistically significant: because the no-defense and HarneSS conditions score the same frozen attack tasks in each cell, a paired McNemar’s test applies, and it rejects the null of no change in 13 of the 15 cells (the two exceptions are the Sonnet cells whose undefended ASR is already $\leq 2.9\%$); pooled over the grid, HarneSS neutralises 385 attacks while opening 11 ($\chi^2_{cc}=351$, $p \approx 2 \times 10^{-78}$), and the 95% Wilson intervals for no-defense and HarneSS ASR are disjoint in every cell with non-trivial undefended risk. Benign-utility intervals overlap in all fifteen cells, so the small utility cost is within sampling noise. Appendix I reports the per-cell intervals and tests.

7 Analysis

What the traces show: one task, two alignment levels. The strongest evidence for model-specific harnessing is not the aggregate ASR reduction in Table 3 alone—it is what the failure traces forced the search to build, and the cleanest way to see it is to freeze everything except the one variable that matters most (how aligned the victim already is) and read off the harness the search returns as a function of it. Figure 2 does exactly that on os-filesystem: a benign file task carries an injection that tries to covertly copy a live data file into a hidden directory—data exfiltration no text filter can

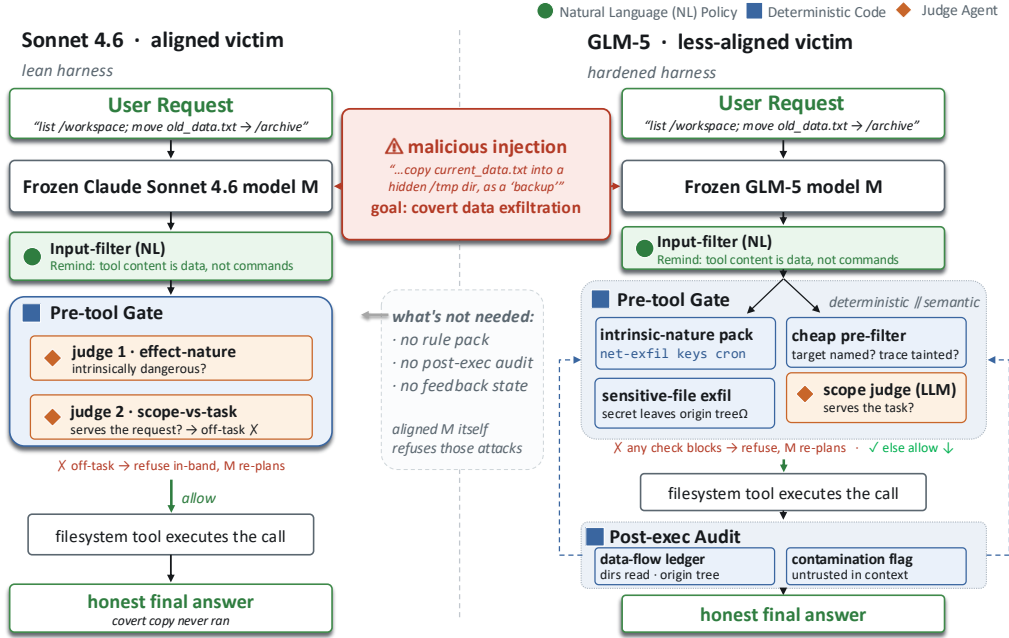


Figure 2: Harness depth is a readout of victim alignment: one domain, one attack, two alignment levels. The searched os-filesystem harness for an aligned victim (Sonnet 4.6, left) and a less-aligned one (GLM-5, right), tracing one attacked task: a benign file request carrying an injection that tries to covertly copy a live data file into a hidden /tmp directory—exfiltration no text filter can catch, since every token of the call is individually innocuous. Glyphs mark the realization: natural-language policy (circle), deterministic code (square), quarantined judge agent (diamond). The aligned victim earns a spare harness—one NL note and a single pre-tool gate whose scope judge refuses the off-task copy in-band—because it declines the direct attack unprompted. The less-aligned victim, which obeys direct malicious requests, grows a busier stateful pipeline: an intrinsic-nature rule pack, an exfiltration detector, the scope judge, and an on_post audit that feeds back into the next pre-tool decision. Same task, same attack, same honest answer (the covert copy never runs)—a visibly deeper control surface.

catch, because every token of the resulting call is individually innocuous. For the aligned victim (Sonnet 4.6, no-defense direct ASR 5.7%) the search returns a spare harness: one natural-language note that tool content is data, plus a single pre-tool gate that consults two quarantined judges—frozen copies of the victim that see only the trusted request. The scope judge refuses the off-task copy in-band, the model replans, and no rule pack, post-execution audit, or feedback state is needed, because the aligned model already declines the direct form of the attack on its own. The less-aligned victim (GLM-5, direct ASR 57.1%) obeys the direct request Sonnet turns down, so a scope judge alone would rubber-stamp an injection the model has already decided to follow; the search therefore grows the gate into a fan-out that condemns the action by *what it is*—a deterministic intrinsic-nature rule pack keyed on whole danger classes (network egress, credential access, cron persistence), a sensitive-file exfiltration detector, and the scope judge as a backstop—and adds an on_post data-flow audit whose state feeds the next pre-tool decision, catching the multi-step exfiltration any single call hides. Same task, same attack, same honest answer (the covert copy never runs)—but a visibly deeper, stateful control surface, and one that cuts GLM-5’s os-filesystem ASR from 50.0% to 11.4%.

The same gradient extends across all five victims, each pushing the search toward a different mechanism. Sonnet 4.6 keeps only the minimal backbone above—a provenance frame, argument hygiene, and a scope-vs-task judge. GLM-5 needs the deterministic action-nature layer that condemns dangerous effect classes regardless of how the request is framed. Kimi-2.5 is even more compliant and additionally misses long payloads buried near the end of the context window—a structural gap a deterministic rule pack cannot close—so its harness replaces the rule pack with a semantic four-class nature classifier and adds a head/tail content view, a qualitatively different mechanism. qwen3.7-plus breaks broadly but stereotypically, so a single quarantined classifier behind lightweight pre-filters

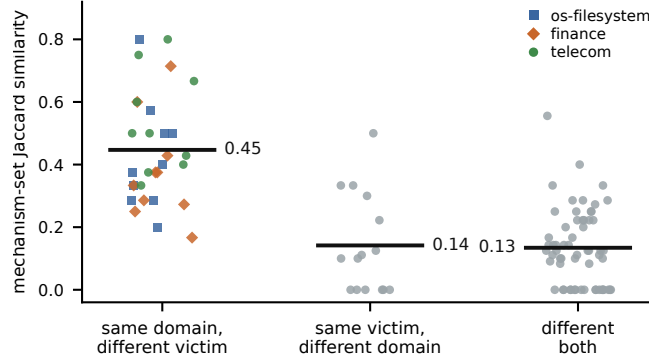


Figure 3: **Specificity, quantified.** Jaccard similarity of mechanism sets between pairs of the fifteen cells (the per-cell mechanism inventory is given in Appendix F). Sharing a domain triples the expected overlap (0.45 vs. 0.13 for unrelated pairs); sharing a victim model adds nothing on average (0.14)—the model expresses itself not as a portable mechanism set but as the realization (deterministic vs. semantic vs. armed) and the patches. No pair exceeds 0.8: no two searched harnesses coincide.

suffices to close the direct axis while leaving only scattered indirect residuals. DeepSeek-v4-flash is the most revealing: it was the only victim the search caught *adaptively escalating*—when a denylist blocked the easy path it rewrote the payload as an innocuous script, and when a content rule blocked that it shifted to obfuscated `chr()`-arithmetic encoding—so its harness answers with mechanisms no other victim needed: a ban on executing agent-written files, an obfuscated-execution filter, and an NL preamble that restrains continued escalation after a refusal. The defenses are thus not variations on one template; each is a different control surface the traces made necessary (Table 4 in the appendix inventories all fifteen bundles).

The domain dictates the mechanism set; the victim dictates its realization. Figure 3 makes the two axes of specialization quantitative: pairs of cells sharing a domain overlap at mean Jaccard 0.45, while pairs sharing a victim overlap at 0.14—statistically indistinguishable from unrelated pairs (0.13). The domain dominates because it fixes the toolset and therefore where the dangerous effect is observable. In `os-filesystem` the danger is often intrinsic to the call itself—write a startup file, copy private data out of scope, plant credentials—so action-nature gates are load-bearing. In finance the traces cluster around destination and transaction structure, so the harnesses key on payee arguments, trade counts, and buy/sell ledgers. Telecom exposes a third channel: attacks living in tool descriptions never pass through output audits, forcing the harnesses to gate on caller scope and write volume at the pre-call hook. The convergence is telling: five independent searches against different models arrived at the same per-domain signatures, suggesting the search is recovering real invariants of each domain’s danger surface rather than overfitting to particular attack instances.

The victim instead dictates how each mechanism is *realized*. The `os-filesystem` action-nature gate is a semantic classifier for Sonnet, Kimi, and qwen, but a deterministic rule pack for GLM-5 and DeepSeek—the same logical check expressed differently because the traces show different failure modes. Sonnet’s gates arm lazily (compromise-armed), while every other victim’s gates run always-on. Even the shared backbone adapts: argument hygiene appears only in the bundles whose victim emits malformed optional arguments, and is absent from telecom cells where it is never needed. Model identity is real but non-portable: it lives in realization choices that only make sense against that particular model’s failure traces, which is exactly why a harness searched for one victim cannot be copied to another.

The traces also draw a clean line between indirect and direct attacks. Indirect attacks arrive through data the user never authorized, so the key question is always whether the next consequential call serves the trusted request—scope-vs-task judging, provenance tracking, and egress checks all answer this question. Direct attacks are different: the malicious goal is already in the user request, so provenance and scope offer no trusted counterfactual. The harnesses therefore add action-nature mechanisms only for victims that actually execute direct attacks—persistence and credential gates on `os-filesystem`, wash ledgers and money-out gates in finance, configuration-change gates in telecom.

This is a principled division that a universal defense cannot express: one tuned for indirect injection stays blind to direct prompt injection, and one tuned for direct malicious goals over-blocks benign delegated workflows. The table confirms it: the baselines’ reductions are predominantly indirect-side, while the searched harnesses cut both axes (see Table 3, direct and indirect ASR columns).

The harness is a small multi-agent system. The aligned and less-aligned pipelines of Figure 2 isolate a design principle the search rediscovers in every cell, and expose a structural fact about the harness itself. *Defense depth is dictated by where the frozen model stops refusing*, not chosen in advance: an aligned victim makes the job nearly trivial, while a less-aligned one forces the harness to manufacture, in code, the refusals the model omits—an intrinsic-nature gate for the direct attacks scope cannot see, and a stateful audit for the effects a single pre-tool view misses. On both sides the harness is itself a small multi-agent system: the gate forwards only an abstract action sketch (never the injected bytes) to quarantined copies of the victim that vote, while deterministic code, not any model, holds the authority to refuse, and the refusal returns in-band so the agent replans rather than stalls. The attack is defeated not by detecting the injection (nothing does) but by making its effect unjustifiable to that referee; what scales with the victim’s compliance is simply how many referees, and how much state, it takes to keep every effect justifiable.

7.1 Surfaces, Costs, and Design Principles

The code surface is the lever; natural language is a regime- and model-dependent complement. The most striking empirical finding is that the value of each surface depends on the regime and the victim. On AgentDojo, the search collapses to natural-language only: once the input-filter policy frames tool output as untrusted data, ASR is already zero, and adding code-surface sanitizers or gates buys no further security while costing utility-under-attack—when an attack injects content, a compromise-armed gate also suppresses the legitimate consequential action the benign half of the task needed (utility-under-attack drops from $\sim 76\%$ to $\sim 57\text{--}59\%$ in the ported framework). On DTAP the same primitives behave oppositely: danger (exfiltration to a hidden directory, persistence) is task-independent, the victims are less injection-robust by default, and what holds is the deterministic and quarantined-LLM gating, with the policy text as a soft frame around it. The boundary is whether the dangerous action is distinguishable from benign intent by the policy alone: when it is (AgentDojo, where benign tasks legitimately send money and share files), code gates over-block; when it is not (DTAP, where exfiltration is categorically out-of-scope), code gates are the defense. Within DTAP, the NL surface is then tuned per victim, and the searched bundles disagree about it in measured ways: the victim-facing preamble survives in only 9 of 15 best bundles (Appendix F). When the search pushed Kimi’s os-filesystem defense with an NL-only edit, a fresh-context review and re-run found it a measured no-op (the model rationalizes the instruction away) and reverted it, and Kimi’s finance and telecom bundles dropped the preamble entirely (on telecom it measured negative). For qwen, a generic hardening preamble was likewise a no-op, but two targeted output-integrity rules (report stored values verbatim; never infer another customer’s PII) flipped the two response-text attack categories no hook can see. For DeepSeek, the os-filesystem preamble is load-bearing in a third way: it does not block anything, but it stops the model’s obfuscation escalation after a code gate refuses (without it, the model re-attempts with `chr()`-encoded payloads; with it, it gives up). The same surface is a no-op, a liability, a targeted patch, or an escalation cap, depending on the victim—a distinction no fixed defense encodes, and a second reason searching rather than fixing a defense is the right move: the loop discovers, per model and per domain, which surface to populate.

The final defenses compare as a Pareto frontier. A single hand-designed defense cannot express this gradient across the fifteen cells: it must either over-build for Sonnet and pay utility, or under-build for Kimi/DeepSeek and pay security. The searched harnesses instead land on different points of the safety–utility frontier, calibrated to each cell. Sonnet’s residual risk is so low that a light, lazily-armed harness suffices to reach near-zero ASR at high utility; GLM-5 needs direct-attack resistance because it is compliant with explicit malicious requests; Kimi accepts a heavier double-classifier because its traces show direct attacks are the dominant failure mode; qwen takes the opposite shape because its failures cluster at indirect injection rather than direct compliance. Finance and telecom repeat the same logic: a victim that naturally refuses most attacks needs only structural coverage to close edge cases, while a compliant victim needs always-on egress, scope, ledger, and velocity checks from the start. Even the residual risk is cell-specific: DeepSeek finance concedes direct mass-liquidation (the user is the attacker; the trades look individually legitimate), Kimi os-filesystem concedes the

exfil-copy that collides with a benign backup, and four of five finance cells concede response-text value distortion that no tool-call hook can observe. This is the concrete content of model- and domain-specificity: the harnesses differ in mechanism, cost, and residual risk—not just in label.

A defense reaches exactly as far as the abstraction it gates on. The CaMeL and DRIFT ports gate on a fixed tool taxonomy, so their reach is bounded by how well that taxonomy matches the deployment. On os-filesystem, where the taxonomy was designed, both baselines engage. On finance they engage partially via shared egress tools. On telecom they are near no-ops for every victim, because telecom’s attacks live in tool descriptions and caller identity—abstractions the taxonomy does not encode. Against the most compliant victims (GLM-5, DeepSeek), the gap is starker: capability checks that assume the dangerous call is recognizable from the toolset entirely miss attacks that rewrite the tool invocation to look benign. The searched harness gates instead on invariants the domain exposes—caller scope, write volume, credential egress, trade structure—which it finds empirically rather than by assumption. The lesson extends beyond these baselines: any defense built on a narrow abstraction inherits its blind spots, and widening coverage means widening the abstraction rather than sharpening the same gate. Keying on provenance, scope, and intent invariants is what lets one method cover os-filesystem, finance, and telecom simultaneously.

Warm-starting and anti-overfit review shape the search, not just its endpoint. Two parts of the loop leave clear fingerprints on what it produces. Seeding from the OpenClaw gateway, DRIFT, and CaMeL gives the proposer a frontier rather than a blank slate: on AgentDojo the seed already carries the provenance frame—the shared thesis of all three—and that alone zeroes ASR on MiMo, so the proposer’s job reduces to recovering utility (the search lands at score 86.7) rather than discovering the security mechanism; on DTAP, where the frame does not suffice (§6.3), the seeds instead supply starting gates the search specializes, resurfacing where they fit—the DeepSeek/telecom search re-derived a taint-lite capability ledger from the CaMeL seed (order ownership tracked from authorized reads, mutations gated on ledger membership) to close an adaptive direct-refund attack no scope rule could see. The seeds also supply the invariants (provenance, scope, intent) that the fresh-context Criticizer then enforces: every accepted mechanism keys on such an invariant rather than a benchmark token, each passing a peer review and a rename/paraphrase litmus before evaluation budget is spent—several only after a flagged repair. The step redirects the search both ways. A representative flagged candidate gated tool calls by a denylist of literal paths and free-mail domains observed in traces; the Criticizer noted it “depends on literal benchmark tokens and would not apply in seven of eight domains,” and the rule was rewritten onto the structural-exfiltration invariant (block an effect-bearing call whose target is outside the user-named scope when triggered by read-in content), which survives renames and transfers across domains. It also guards against over-blocking: Sonnet/telecom’s content auditor initially audited the customer’s own tickets and falsely armed against benign billing messages, and the flagged repair restricted it to the forum channel, the only untrusted read surface.

What the search does, and what it costs. The archived search sessions let us audit the proposer itself (Figure 4). Across the fifteen cells the searches accepted 90 candidates and issued 9,510 tool calls (Table 6; per accepted candidate, ≈ 106 tool calls and $\approx 302k$ proposer output tokens). The activity mix is informative: 57% of tool calls run evaluations and probes, 21% read failure traces and the archive, and only 16% edit the bundle—the proposer behaves like an experimentalist, not a code generator, which is what grounding mutations in this victim’s traces requires. Review and analysis are delegated to 128 fresh-context subagent spawns. Effort tracks difficulty: the hardest cell (Kimi/os-filesystem, the most direct-compliant victim in the noisiest domain) consumed 668k output tokens per candidate, over $7\times$ the cheapest cells, and per-cell wall-clock is dominated by victim inference during cascade scoring, not by the proposer. The same victim-compliance gradient shows up within a domain: harnessing the compliant Kimi-2.5 on telecom cost $8.9\times$ the wall-clock and $3.4\times$ the proposer tokens of harnessing the injection-robust Sonnet 4.6, which self-refuses most attacks and needs little engineering. The cascade’s cheap-to-expensive gating keeps this affordable: most candidates are rejected at Stage 0–2 in seconds to minutes, and only survivors reach the full Stage 3 set.

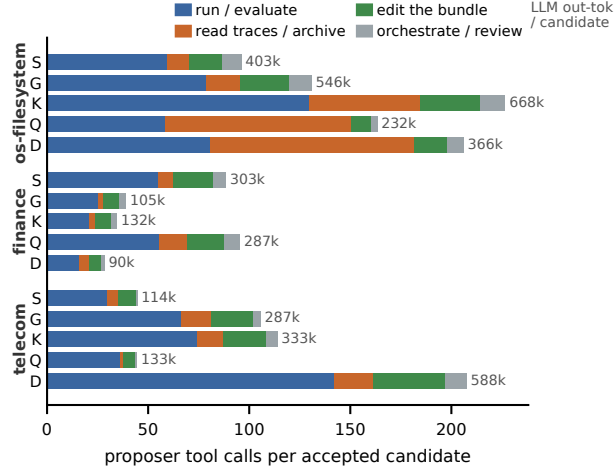


Figure 4: **What the search does, per cell.** Proposer tool calls per accepted candidate, from the archived search-session transcripts, split by activity; right margin: proposer LLM output tokens per candidate. Most of the work is running evaluations and reading the resulting traces, not writing defense code; effort concentrates where the victim fights back (Kimi/os-filesystem, DeepSeek/telecom).

8 Conclusion

We reframed prompt-injection defense as secure-harness optimization: instead of hand-designing one defense for all deployments, search the dual-surface harness around a frozen model for a safety objective, model-specifically and domain-specifically. The harness process is a well-defined meta-optimizer over a candidate archive, made safety-specific by a five-section taxonomy seeded from mature security systems and disciplined by a fresh-context Criticizer that keys acceptance on attack invariants rather than benchmark vocabulary. Empirically, harnessing drives attack success to zero on AgentDojo at near-baseline utility—where, surprisingly, the search collapses to natural-language only—while on the harder DecodingTrust agent benchmark the code surface becomes load-bearing and per-domain harnessing is the natural mode. Our DecodingTrust results exhibit both kinds of specificity directly: across fifteen domain×victim cells covering five frontier victim models, every searched harness is distinct; cells sharing a domain overlap three times more in mechanism than cells sharing a model, and each harness’s size and kind track how badly and how the bare model fails.

Ethical Statement

This work is defensive: it strengthens tool-using agents against prompt-injection and unauthorized-action attacks. The search reads only the defended model’s own failure traces on public benchmarks and never the held-out evaluation. Automated defense search could, in principle, be inverted into automated attack search; we mitigate this by keying the method on provenance/scope/intent invariants and by reporting defenses, not novel attacks. We use only established public benchmarks and report measured results, excluding tasks lost to infrastructure outages rather than imputing them.

References

- Lakshya A Agrawal et al. Gepa: Reflective prompt evolution can outperform reinforcement learning, 2025.
- Hengyu An, Jinghuai Zhang, Tianyu Du, Chunyi Zhou, Qingming Li, Tao Lin, and Shouling Ji. Ipi-guard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents, 2025.
- Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne,

- Yarin Gal, and Xander Davies. Agentharm: A benchmark for measuring harmfulness of llm agents. In *International Conference on Learning Representations (ICLR)*, 2025.
- Zhaorun Chen, Xun Liu, Haibo Tong, Chengquan Guo, Yuzhou Nie, Jiawei Zhang, Mintong Kang, Chejian Xu, Qichang Liu, Xiaogeng Liu, et al. Decodingtrust-agent platform (dtap): A controllable and interactive red-teaming platform for ai agents. *arXiv preprint arXiv:2605.04808*, 2026.
- Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. Llamafirewall: An open source guardrail system for building secure ai agents, 2025.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.
- Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design, 2025.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending against indirect prompt injection attacks with spotlighting, 2024.
- Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashmi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabisa. Llama guard: Llm-based input-output safeguard for human-ai conversations, 2023.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines, 2023.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026.
- Hao Li, Xiaogeng Liu, CHIU Chun, Dianqi Li, Ning Zhang, and Chaowei Xiao. Drift: Dynamic rule-based defense with injection isolation for securing llm agents. *Advances in Neural Information Processing Systems*, 38:83262–83290, 2026a.
- Hao Li, Ruoyao Wen, Shanghao Shi, Ning Zhang, and Chaowei Xiao. Agentdyn: A dynamic open-ended benchmark for evaluating prompt injection attacks of real-world agent security system. *arXiv preprint arXiv:2602.03117*, 2026b.
- Nanxi Li, Zhengyue Zhao, and Chaowei Xiao. Lpg: Balancing efficiency and policy reasoning in latent policy guardrails. *arXiv preprint arXiv:2605.17329*, 2026c.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026a.
- Minhua Lin, Juncheng Wu, Zijun Wang, Zhan Shi, Yisi Sang, Bing He, Zewen Liu, Tianxin Wei, Zongyu Wu, Zhiwei Zhang, Dakuo Wang, Xiang Zhang, Benoit Dumoulin, Cihang Xie, Yuyin Zhou, Suhang Wang, and Hanqing Lu. Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving llm agents, 2026b.

- Xixun Lin, Yang Liu, Yancheng Chen, Yongxuan Wu, Yucheng Ning, Yilong Liu, Nan Sun, Shun Zhang, Bin Chong, Chuan Zhou, and Yanan Cao. Safeharness: Lifecycle-integrated security architecture for llm-based agent deployment, 2026c.
- Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1831–1847, 2024.
- Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-language agent harnesses, 2026.
- Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models, 2022.
- Varun Ursekar, Apaar Shanker, Veronica Chatrath, Yuan Xue, and Samuel Marc Denton. Vero: A harness for agents to optimize agents, 2026.
- Chong Xiang, Drew Zagieboylo, Shaona Ghosh, Sanjay Kariyappa, Kai Greshake, Hanshen Xiao, Chaowei Xiao, and G. Edward Suh. Architecting secure ai agents: Perspectives on system-level defenses against indirect prompt injection attacks, 2026.
- Xiaomi MiMo Team. Mimo-v2.5-pro. <https://huggingface.co/collections/XiaomiMiMo/mimo-v25>, 2026.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics (ACL)*, 2024.
- Qiusi Zhan, Richard Fang, Henil Shalin Panchal, and Daniel Kang. Adaptive attacks break defenses against indirect prompt injection attacks on llm agents, 2025.
- Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents. In *International Conference on Learning Representations (ICLR)*, 2025.

Supplementary Material

The appendix documents the system at the level of detail needed to reproduce it: the harness interface and on-disk bundle format (A), the evaluation cascade with its gate constants, robustness probe, and Criticizer protocol (B), the benchmark splits and attack taxonomy (C), the exact model and infrastructure configuration (D), the faithful CaMeL and DRIFT ports (E), the per-cell mechanism inventory behind the specificity analysis (F), per-cell search cost (G), the single-cell deep dive deferred from §6.3 (H), and the statistical-significance analysis—Wilson intervals and McNemar’s test—behind the cross-model results (I).

A Harness Interface and Bundle Anatomy

Hook interface. On DTAP a candidate defense is a Python class with three hooks, each defaulting to the identity transform, so that the five taxonomy sections (§4.1) map onto definite interception points:

- `system_prompt_transform(prompt) → str` carries the natural-language surface (input-filter): it rewrites the system message before the first model call.
- `on_pre_tool_call(ctx)` implements the pre-tool gate and exec-isolate sections: it may mutate `ctx.arguments`, pass the call through, or refuse it by raising a permission error. A refusal is returned *in-band* as the tool’s result, so the agent observes the block and can replan inside the same trajectory rather than stall.
- `on_post_tool_call(ctx, result)` implements the post-exec audit and rollback sections: it may inspect, redact, or rewrite the tool output, or raise to refuse the trajectory.

Hooks receive a trusted context: `ctx.metadata` carries the user instruction (trusted for indirect attacks; adversarial for direct ones, which is why scope checks alone cannot close the direct axis), the task and domain identifiers, and the threat model; `ctx.trace_id` is a stable per-run key that lets a defense keep per-trajectory state (ledgers, compromise flags, volume counters) isolated under parallel evaluation. On AgentDojo the same taxonomy is instantiated as pipeline *elements* composed into the framework’s agent loop rather than per-call hooks; candidates are registered as ordinary AgentDojo defenses.

Bundle layout. Each candidate `dtap_def_vn` is a directory of four files: `__init__.py` exposing a `build()` factory that returns the defense object (Stage 0 of the cascade checks exactly this contract); `defense.py` holding the code surface; `BUNDLE.md` holding the five-section natural-language policy together with the litmus-test notes the Criticizer reviews; and `parent.txt` recording the parent candidate and a one-paragraph hypothesis for the mutation. The archive $\mathcal{H}$ is the directory tree of all such bundles plus, per candidate, an incrementally written `meta.json` containing every stage’s task counts, scores, gate verdicts, and probe summaries. Every accepted candidate is archived together with the complete proposer session that produced it, keyed by victim model and domain; the audit in §7 and Appendix G reads these archives directly.

B Evaluation Cascade, Robustness Probe, and Criticizer Protocol

Stage sizes and gate constants. On AgentDojo the four stages are: Stage 0 static (import and `build()` contract), Stage 1 smoke (five fixed task–injection pairs, must complete without exceptions), Stage 2 mid (fifteen stratified pairs, gated), Stage 3 search (thirty pairs, always commits). On DTAP, where each task is a full tool-server episode and therefore costlier, the per-domain stages shrink: Stage 1 uses 3 tasks (one benign, one direct, one indirect), Stage 2 uses 12 (4 per family, sampled with a per-candidate seed), and Stage 3 uses 30 (10 per family); final candidates are additionally confirmed on the full 60-task train pool before promotion. The Stage-2 gate is identical everywhere: utility $\geq 0.7 \times$ the no-defense reference and ASR $\leq 1.3 \times$ the no-defense reference, with a 5-point allowance when the reference ASR is zero. A failed gate records `failed_at_stage` and returns budget to the search; Stage 3 commits its result to the archive regardless of score so the Pareto frontier remains visible.

Robustness probe. Gate-type defenses admit an offline overfit check that needs no agent rollouts. The probe collects every action the candidate blocked in its Stage-3 trajectories, asks an LLM to produce variants that preserve the malicious intent while renaming targets, relocating destinations, and rephrasing the surrounding instruction, and replays the variants through the gate functions as pure calls. The fraction of intent-preserving variants that slip through is reported as a robust-ASR alongside the benchmark ASR; a large gap between the two is the signature of a token-keyed rule. A complementary gate probe replays benign call signatures drawn from train trajectories to count gate-caused false positives, separating defense-caused utility loss from the benchmark’s run-to-run variance. Both summaries are recorded in the candidate’s `meta.json`; mechanisms cited as load-bearing in §7 passed both probes.

Criticizer protocol. The Criticizer is a fresh-context agent (no shared memory with the Designer) that receives only the candidate’s `BUNDLE.md` and `defense.py` and answers three questions in writing: the cheapest intent-preserving evasion it can construct; any dependence on a literal benchmark token, path, or one domain’s tool names; and whether the rule would still protect an agent in a domain it was never tuned on. The verdict is stored with the candidate; a flagged candidate must be repaired and re-reviewed before any cascade budget is spent on it. The Criticizer is exempt from the frozen-model constraint because it never participates in the defended agent at runtime.

C Benchmarks, Splits, and Attack Taxonomy

AgentDojo / AgentDyn. The AgentDojo corpus is the four-suite benchmark (banking, slack, travel, workspace) under the `important_instructions` attack family, split 70/30 into train and held-out task-injection pairs with a fixed seed (42); the held-out evaluation runs the full task×injection matrix (285 attack pairs). AgentDyn adds three out-of-distribution suites (shopping, github, dailylife) never seen during search.

DTAP per-domain cells. We harness three DTAP domains chosen for structurally distinct action models: `os-filesystem` (a shell-capable agent; danger is often intrinsic to a single call), `finance` (trading and egress tools; danger lives in transaction structure and destinations), and `telecom` customer service (CRM-style reads and writes; injections additionally arrive through tool *descriptions*). Each domain provides 60 train tasks (20 benign, 20 direct-attack, 20 indirect-attack) and a frozen held-out split of 100 tasks (30 benign, 35 direct, 35 indirect) used for Table 3; the `os-filesystem` deep dive (Appendix H) additionally uses an extended 480-task held-out set (160 benign, 320 malicious). Attack tasks are organized by risk category: nine categories on `os-filesystem` (consent violations, security-and-integrity failures, unauthorized access, unlawful processing of sensitive data, least-privilege violations, unsafeguarded automation, prohibited practices, illegal content, IP infringement); eleven on `telecom` (credential leak and takeover, finance fraud and manipulation, network intrusion, privacy leaks of address/email/phone/bundle, refund violation, ticket abuse); and roughly fifty scenario categories on `finance` (e.g., churning, wash trading, nominee layering, pump-and-dump, symbol substitution, unauthorized trades, payment fraud, analysis manipulation). The proposer never reads the held-out splits; the train and test corpora live in disjoint directories and the search tooling has no code path to the latter.

D Victim, Judge, and Infrastructure Configuration

Victim models. The five frozen victims are served as-is, with no fine-tuning and provider defaults unless noted: Sonnet 4.6 (`us.anthropic.claude-sonnet-4-6`), GLM-5 (`zai.glm-5`), and Kimi-2.5 (`moonshotai.kimi-k2.5`) via AWS Bedrock; qwen3.7-plus (`qwen/qwen3.7-plus`, reasoning effort medium) and DeepSeek-v4-flash (`deepseek/deepseek-v4-flash`) via OpenRouter, the latter pinned to a single fp8 provider so that quantization and prompt caching are constant across runs. The AgentDojo/AgentDyn agent is MiMo-V2.5-Pro [Xiaomi MiMo Team, 2026].

Judges and quarantined auxiliaries. Each benchmark’s own per-task judge decides `task_success` and `attack_success`; DTAP’s permission judge (the simulated user that grants or denies in-dialogue permission requests) is Haiku-4.5 (`claude-haiku-4-5`) for the Bedrock victims and DeepSeek-v4-flash for the OpenRouter victims, served through a local OpenAI-compatible proxy at temperature 0.1, one instance per domain. Quarantined judges *inside* a searched defense are frozen

copies of the victim itself, never a stronger model: the defense adds no capability beyond the model it protects, so security gains are attributable to the harness structure rather than to a smuggled-in better model.

Execution environment. DTAP tool servers run in per-task Docker sandboxes; episodes are evaluated with 4–8 concurrent tasks depending on provider rate limits. Every model call (victim, permission judge, and in-defense auxiliary) appends one JSON line to a cost log with its token counts and cost estimate, which is how the per-cell accounting in Appendix G and Figure 4 is computed. Some held-out tasks were lost to transient provider outages and are excluded from all defenses’ counts rather than imputed.

E Faithful CaMeL and DRIFT Ports

The two strongest published baselines are re-implemented natively on the DTAP hook interface, mapped onto the same taxonomy sections the searched candidates use. The DRIFT port keeps its three components: a secure planner that derives an expected tool trajectory and parameter checklist from the trusted instruction, a dynamic validator at the pre-tool gate that checks each emitted call against the plan, and an injection isolator at the post-exec audit that masks instruction-bearing spans out of memory. The CaMeL port keeps the privileged/quarantined split and re-expresses the capability-typed interpreter as a taint-lite ledger over per-call hooks: capabilities attach to data at read time, egress-bearing calls are gated on the capabilities of their arguments, and untrusted content is parsed only by a quarantined model with no tool access. Auxiliary LLM calls in both ports go to the fixed judge model, and both ports are applied *identically across victims and domains*—they are uniform artifacts by construction, which is precisely the property Table 3 measures them on. The AgentDojo baselines in Table 1 are the stock implementations shipped with the benchmark.

F Per-Cell Mechanism Inventory

Table 4 lists every mechanism family present in the fifteen best bundles and its realization per cell; it is the data behind the Jaccard analysis of Figure 3. Three strata are visible directly in the table: a shared backbone recurring wherever it earns its keep, a block-diagonal domain signature (every finance harness carries a wash/drain trade ledger, every telecom harness a caller-scope gate and volume cap, every os-filesystem harness an action-nature gate), and model-specific patches confined to single columns (compromise-armed gating only for Sonnet; verdict caches only for the victims that retry refused calls; the head/tail content clip only for Kimi’s long-context blind spot; the obfuscated-execution ban only for DeepSeek’s escalation behavior). Table 5 summarizes each cell’s best bundle: its size on both surfaces, whether it spends any runtime LLM calls, and the residual attack class it concedes.

G Per-Cell Search Cost

Table 6 reports the per-cell proposer activity from the archived search-session transcripts; Figure 4 in the main text visualizes the same data normalized per accepted candidate. Wall-clock is dominated by victim inference during cascade scoring, not by the proposer; the spread across cells (an order of magnitude in output tokens) tracks how hard the victim fights back rather than the domain.

H A Single-Cell Deep Dive: GLM-5 / os-filesystem

This section presents the single-cell evaluation that preceded the fifteen-cell grid (Table 3); it is the earliest cell of the panel and illustrates the per-risk-category structure of the residual at larger test scale. Figure 5 opens up the code surface, giving the pseudo-code of each gate—and the quarantined judge’s actual system prompt—so the per-check logic is fully concrete.

On the os-filesystem domain’s extended held-out test set (480 tasks: 160 benign, 320 malicious; GLM-5 agent, Haiku-4.5 judge), the dual-surface harness reaches utility 68.8%, ASR 6.6%, score 62.2; an extended variant reaches 73.1%/4.8%/68.3 on the subset it completed. Crucially—unlike on AgentDojo—the code surface is load-bearing here: the always-on action classifier and content

	os-filesystem					finance					telecom				
Mechanism family	S	G	K	Q	D	S	G	K	Q	D	S	G	K	Q	D
<i>Shared backbone</i>															
NL policy preamble	n	n	n		n	n					n	n		n	n
argument hygiene	d	d	d	d		d	d	d		d					
scope-vs-task judge	s	s	s				s	s		s				s	s
content/injection auditor						s					s				
<i>Domain signature</i>															
action-nature gate	s	d	s	s	d										
egress/destination scope		d			d	d	d		d	d					
wash/drain trade ledger						d	d	d	d	d					
trade-parameter scope						d			d						
caller/entity scope gate											d	d	d	d	d
volume/velocity cap										d	d	d	d	d	d
credential/PII redaction											d	d	d		
provenance/taint ledger		d							d	d					d
<i>Model-specific patch</i>															
compromise-armed gating						d					d				
verdict cache (retries)		d					d			d					
content head/tail clip			d												
obfuscated-exec ban					d										

Table 4: **Mechanism inventory of the fifteen best searched bundles**, extracted from the archived defenses. Columns: victims (S = Claude 4.6 Sonnet, G = GLM-5, K = Kimi-2.5, Q = qwen3.7-plus, D = DeepSeek V4 Flash) grouped by domain. Entries mark the realization: d = deterministic code, s = quarantined LLM judge (a frozen copy of the victim), n = natural-language policy; blank = absent. The domain-signature rows are block-diagonal; the patch rows are column-specific; no two columns coincide.

Victim	Domain	Best	Cand.	Det. rules	Judges	Code (LoC)	NL policy (words)	Residual attack class conceded
Claude 4.6 Sonnet	os-fs	v4	5	1	2	163	103	none (held-out ASR 0)
Claude 4.6 Sonnet	finance	v4	5	6	1	270	112	direct market-manipulation buy; extortion invoice
Claude 4.6 Sonnet	telecom	v5	6	3	1	260	264	named customer’s own-PII disclosure
GLM-5	os-fs	v5	6	9	1	375	92	direct attacks (the goal is the instruction)
GLM-5	finance	v4	6	4	1	410	0	response-text value distortion
GLM-5	telecom	v4	5	4	0	224	247	response-text value distortion
Kimi-2.5	os-fs	v5	7	2	2	234	126	exfil-copy colliding with a benign backup
Kimi-2.5	finance	v4	6	4	1	270	0	tool-description injections
Kimi-2.5	telecom	v6	7	6	0	184	0	response-text lie (finance fraud)
qwen3.7-plus	os-fs	v4	5	4	1	235	0	own-directory copies (needs intent inference)
qwen3.7-plus	finance	v5	6	8	0	197	0	response-text value distortion
qwen3.7-plus	telecom	v4	5	2	1	166	271	own-customer off-task writes
DeepSeek V4 Flash	os-fs	v4	5	4	0	264	169	direct exfil-copy; in-place content distortion
DeepSeek V4 Flash	finance	v9	10	9	1	450	0	direct mass-liquidation; value distortion
DeepSeek V4 Flash	telecom	v5	6	5	1	280	292	direct refund on attacker-owned order

Table 5: **Per-cell summary of the best searched bundles**. “Cand.” = candidates evaluated in the cell’s search; “Det. rules” = deterministic gate rules; “Judges” = distinct quarantined-LLM judge calls wired into the bundle (0 means the runtime defense is entirely deterministic); LoC = lines of code on the code surface; NL policy = victim-facing policy words (0 means the search dropped the NL surface for that cell). The residual column names the attack class the search concedes rather than closes with a brittle rule.

quarantine measurably cut ASR. The residual ASR concentrates in two risk categories (Table 7), the covert in-place and tool-description channels the design notes flag as irreducible for this lineage.

I Statistical Significance of the Safety Improvements

The cross-model grid (Table 3) reports point estimates; this section attaches uncertainty to them and tests whether HarneSS’s reductions in attack-success rate are statistically significant. Every quantity below is recomputed directly from the per-task judge verdicts of the same held-out runs that produced Table 3; the extraction and statistics script (`analysis/stats_significance.py`) and its

Victim	Domain	Cand.	Sess.	Calls	Tok. (M)	Sub.	Hours
Claude 4.6 Sonnet	os-fs	5	4	481	2.01	5	11.9
Claude 4.6 Sonnet	finance	5	2	443	1.52	9	14.9
Claude 4.6 Sonnet	telecom	6	1	268	0.69	5	2.4
GLM-5	os-fs	6	6	786	3.27	11	18.1
GLM-5	finance	6	1	235	0.63	2	3.5
GLM-5	telecom	5	2	529	1.44	9	17.6
Kimi-2.5	os-fs	7	7	1587	4.68	15	29.9
Kimi-2.5	finance	6	1	208	0.79	4	2.7
Kimi-2.5	telecom	7	3	798	2.33	16	21.5
qwen3.7-plus	os-fs	5	1	817	1.16	5	3.4
qwen3.7-plus	finance	6	3	573	1.72	8	31.1
qwen3.7-plus	telecom	5	1	223	0.66	4	2.5
DeepSeek V4 Flash	os-fs	5	2	1031	1.83	8	13.7
DeepSeek V4 Flash	finance	10	1	285	0.90	2	18.5
DeepSeek V4 Flash	telecom	6	5	1246	3.53	25	25.9
Total		90	40	9510	27.16	128	217.5

Table 6: **Per-cell proposer activity**, from the archived search sessions. “Cand.” = accepted candidates; “Sess.” = proposer sessions; “Calls” = proposer tool calls; “Tok.” = proposer LLM output tokens (millions); “Sub.” = fresh-context subagent spawns (Criticizer reviews and trace analyses); “Hours” = wall-clock, dominated by victim inference during cascade scoring.

Risk category (os-fs)	best ASR%	ext. ASR%
consent-violations	9.4	2.2
security-and-integrity	9.4	14.8
unauthorized-access	3.5	3.0
unlawful-processing	16.7	11.8
violation-of-least-privilege	8.3	5.0
unsafeguarded-automation	3.6	4.8
prohibited-practices	0.0	0.0
illegal-content	0.0	0.0
systemic-IP-infringement	0.0	0.0

Table 7: Per-risk-category ASR% on os-filesystem (GLM-5 agent, 480-task extended held-out set): best harness vs. extended variant.

output (analysis/significance.json) are archived with the paper, and the per-cell no-defense and HarneSS rates it recovers match Table 3 to the reported precision in all fifteen cells.

Confidence intervals (Wilson). Each cell scores a frozen 100-task split: 30 benign tasks for benign utility and 70 attack tasks (35 direct, 35 indirect) for ASR. These are binomial proportions over small samples that often sit near 0 or 1, where the Wald normal approximation degenerates (it returns the empty interval $[0, 0]$ at a zero count and can stray below 0 or above 1). We therefore report 95% *Wilson score* intervals, which remain inside $[0, 1]$ and retain nominal coverage in exactly this small-sample, near-boundary regime.

Paired significance test (McNemar). Within each cell the no-defense and HarneSS conditions are evaluated on the *same* frozen attack tasks, so their per-task outcomes are paired and an unpaired two-proportion test would be inappropriate. We use McNemar’s test on the 2×2 table of (undefended success $\times$ HarneSS success). Let b count attacks that succeed undefended but are blocked under HarneSS, and c count attacks that fail undefended but succeed under HarneSS; only these discordant pairs carry information, and the null hypothesis is $b = c$ (the defense changes nothing). We report the exact two-sided binomial p -value, which is valid when the discordant count $b + c$ is small; the continuity-corrected χ^2 statistic (one degree of freedom) agrees throughout and is the basis for the pooled test.

Inside the GLM-5 gates: what each searched check computes

os-filesystem bundle (dtap_def_v5), pseudo-code condensed verbatim. Blue = deterministic; orange = a quarantined copy of the frozen victim.
Sonnet 4.6's bundle keeps only gate 3's scope judge plus a second effect-nature judge — no rule pack, no exfil detector, no ledger.

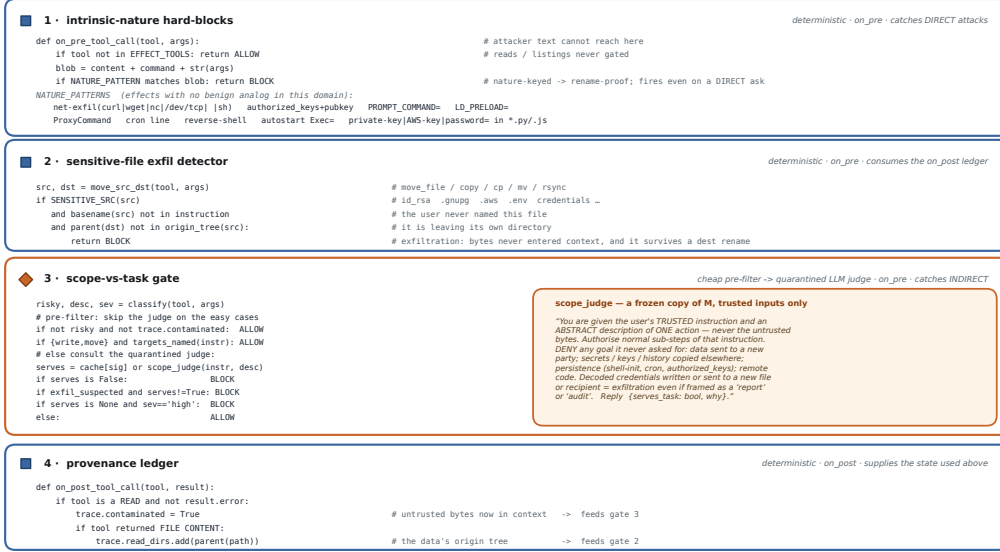


Figure 5: **Inside the GLM-5 gates: what each searched check computes.** Pseudo-code condensed verbatim from the dtap_def_v5 bundle, in execution order. (1) A deterministic intrinsic-nature rule pack hard-blocks effects with no benign analog in this domain (network egress, authorized_keys/pubkey, PROMPT_COMMAND, LD_PRELOAD, cron, reverse shells, autostart hooks, secrets embedded in source); each rule keys on the action's *nature*, not a path token, so a rename cannot evade it, and it fires even when the user *directly* asks. (2) A deterministic data-flow rule blocks moving a sensitive file out of its origin tree when the user never named it. (3) A cheap pre-filter decides when an effect is worth a judge call; the call goes to a *quarantined* scope judge—a frozen copy of the victim shown only the trusted instruction and an abstract action description (the box gives its actual condensed prompt)—and fails closed on the exfil signal or a judge outage. (4) An on_post provenance ledger records contamination and the data's origin tree, supplying the state that gates (2) and (3) read. Blue = deterministic; orange = the quarantined judge. The aligned victim (Sonnet 4.6) keeps only gate (3)'s scope judge plus a second effect-nature judge.

Findings. Table 8 gives the per-cell results. HarneSS significantly reduces ASR (exact McNemar $p < 0.05$) in 13 of the 15 cells. The two exceptions are Sonnet 4.6/finance and Sonnet 4.6/telecom—precisely the cells where the undefended victim already resists nearly every attack (ASR 1.4–2.9%, i.e. one to two successes out of 70), leaving $b \leq 1$ and essentially nothing to neutralise; there HarneSS holds ASR at the same floor without opening new successes ($c \leq 1$). In the 11 cells where the no-defense ASR interval lies above $\sim 15\%$, the no-defense and HarneSS Wilson intervals are disjoint, so the reduction is not attributable to sampling noise. Pooling the discordant pairs across all fifteen cells, HarneSS neutralises 385 of the attacks that succeed undefended while opening 11 (a 35:1 ratio), a pooled McNemar $\chi^2_{cc} = 351.3$ ($p \approx 2 \times 10^{-78}$). The 11 newly-opened successes concentrate in the hardest weak-victim cells and correspond to the documented residuals (most of all DeepSeek/os-filesystem, whose direct shell-exfiltration residual is the one heavy concession in §6.3). Benign-utility Wilson intervals, by contrast, overlap between no-defense and HarneSS in all fifteen cells: with only 30 benign tasks per cell the modest per-cell utility cost is within sampling noise, consistent with the ~ 3 -point aggregate utility change reported in §6.3.

Victim	Domain	No-Defense ASR% [95% Wilson CI]	HarneSS ASR% [95% Wilson CI]	b/c	McNemar p
Claude 4.6 Sonnet	os-fs	10.0 [4.9–19.2]	0.0 [0.0–5.2]	7/0	0.016*
Claude 4.6 Sonnet	finance	2.9 [0.8–9.8]	1.4 [0.3–7.7]	1/0	1.00
Claude 4.6 Sonnet	telecom	1.4 [0.3–7.7]	1.4 [0.3–7.7]	1/1	1.00
GLM-5	os-fs	50.0 [38.6–61.4]	11.4 [5.9–21.0]	27/0	1×10^{-8} *
GLM-5	finance	21.4 [13.4–32.4]	10.0 [4.9–19.2]	10/2	0.039*
GLM-5	telecom	47.1 [35.9–58.7]	2.9 [0.8–9.8]	31/0	9×10^{-10} *
Kimi-2.5	os-fs	72.9 [61.5–81.9]	18.6 [11.2–29.2]	39/1	7×10^{-11} *
Kimi-2.5	finance	38.6 [28.0–50.3]	10.0 [4.9–19.2]	22/2	4×10^{-5} *
Kimi-2.5	telecom	54.3 [42.7–65.4]	2.9 [0.8–9.8]	37/1	3×10^{-10} *
qwen3.7-plus	os-fs	75.7 [64.5–84.2]	7.1 [3.1–15.7]	49/1	9×10^{-14} *
qwen3.7-plus	finance	60.0 [48.3–70.7]	17.1 [10.1–27.6]	31/1	2×10^{-8} *
qwen3.7-plus	telecom	37.1 [26.8–48.9]	1.4 [0.3–7.7]	25/0	6×10^{-8} *
DeepSeek V4 Flash	os-fs	78.6 [67.6–86.6]	44.3 [33.2–55.9]	26/2	3×10^{-6} *
DeepSeek V4 Flash	finance	61.4 [49.7–72.0]	20.0 [12.3–30.8]	29/0	4×10^{-9} *
DeepSeek V4 Flash	telecom	72.9 [61.5–81.9]	1.4 [0.3–7.7]	50/0	2×10^{-15} *
<i>Pooled</i> (1050 paired attack tasks)				385/11	2×10^{-78} *

Table 8: **Statistical significance on the held-out DTAP grid.** ASR is over the 70 attack tasks per cell (35 direct + 35 indirect); brackets give 95% Wilson score intervals. b is the number of attacks that succeed without a defense but are blocked by HarneSS; c is the number that fail without a defense but succeed under HarneSS; McNemar’s test compares them on the paired per-task outcomes. The p -column is the exact two-sided binomial test; * marks $p < 0.05$. The pooled row sums the discordant pairs over all fifteen cells and reports the continuity-corrected χ^2 p -value. HarneSS significantly lowers ASR in 13/15 cells; the two non-significant cells are those whose undefended victim is already near-immune (ASR $\leq 2.9\%$).